```
def faktorial(n):
    hasil = 1
    for i in range(1, n+1):
        hasil *= i
    return hasil

# contoh
print(faktorial(10))
print(faktorial(7))
```

```
3628800
5040
```

```
import sys

def factorial_iterative(n):
    """
    Menghitung factorial menggunakan iterative approach
    Time Complexity: O(n)
    Space Complexity: O(1)
    """
    if n < 0:
        return None
    if n == 0 or n == 1:
        return 1

    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

sys.set_int_max_str_digits(0)


print(f"10177! = {factorial_iterative(10177)}")
print(f"5! = {factorial_iterative(5)}")
print(f"0! = {factorial_iterative(0)}")
```

```
10177! = 136271900197510750886151315442715599519584804953852929871426376681747536360126228608840226875444928879883215518679
5! = 120
0! = 1
```

```
def factorial_recursive(n):
    """
    Menghitung factorial menggunakan recursive approach
    Time Complexity: O(n)
    Space Complexity: O(n) - karena call stack
    """
    if n < 0:
        return None
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)

# Test
print(f"5! = {factorial_recursive(5)}")
```

```
5! = 120
```

```
from decimal import Decimal, getcontext

def factorial_bigint(n):
    """
    Menghitung factorial untuk bilangan besar menggunakan Decimal
    Time Complexity: O(n * log(n)^2) - karena operasi big integer
    """
    getcontext().prec = 10000  # Set precision tinggi

    if n < 0:
        return None
    if n == 0 or n == 1:
        return 1

    result = Decimal(1)
    for i in range(2, n + 1):
        result *= Decimal(i)
    return result

# Test untuk 10177! (akan memakan waktu dan memori besar)
```

```
# print(f"10177! = {factorial_bigint(10177)}")
print(f"10! = {factorial_bigint(10)}")
```

```
10! = 3628800
```

```
def factorial(n):
    """
    Fungsi factorial optimal dengan validasi input
    Time Complexity: O(n)
    Space Complexity: O(1)
    """
    # Validasi input
    if not isinstance(n, int):
        raise ValueError("Input harus bilangan bulat")
    if n < 0:
        raise ValueError("Factorial tidak didefinisikan untuk bilangan negatif")

    # Base case
    if n == 0 or n == 1:
        return 1

    # Iterative calculation
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# Benchmarking
import time

def benchmark_factorial(n):
    start = time.time()
    result = factorial(n)
    end = time.time()
    print(f"{n}! dihitung dalam {end-start:.6f} detik")
    return result

# Test cases
test_cases = [5, 10, 15, 20]
for n in test_cases:
    print(f"{n}! = {factorial(n)}")
```

```
5! = 120
10! = 3628800
15! = 1307674368000
20! = 2432902008176640000
```